Deep Learning for NLP

Large Language Models (LLMs)
Fine-Tuning and Prompting

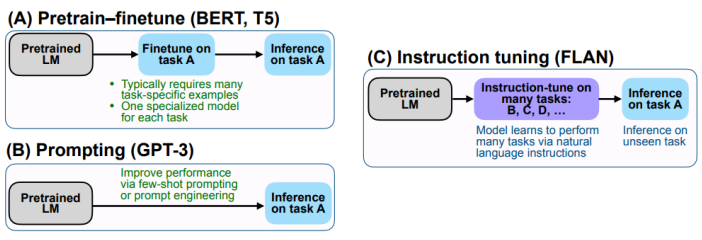


Figure 2: Comparing instruction tuning with pretrain–finetune and prompting.

Learning goals

- comprehend the different subtleties in the space of fine-tuning and prompting

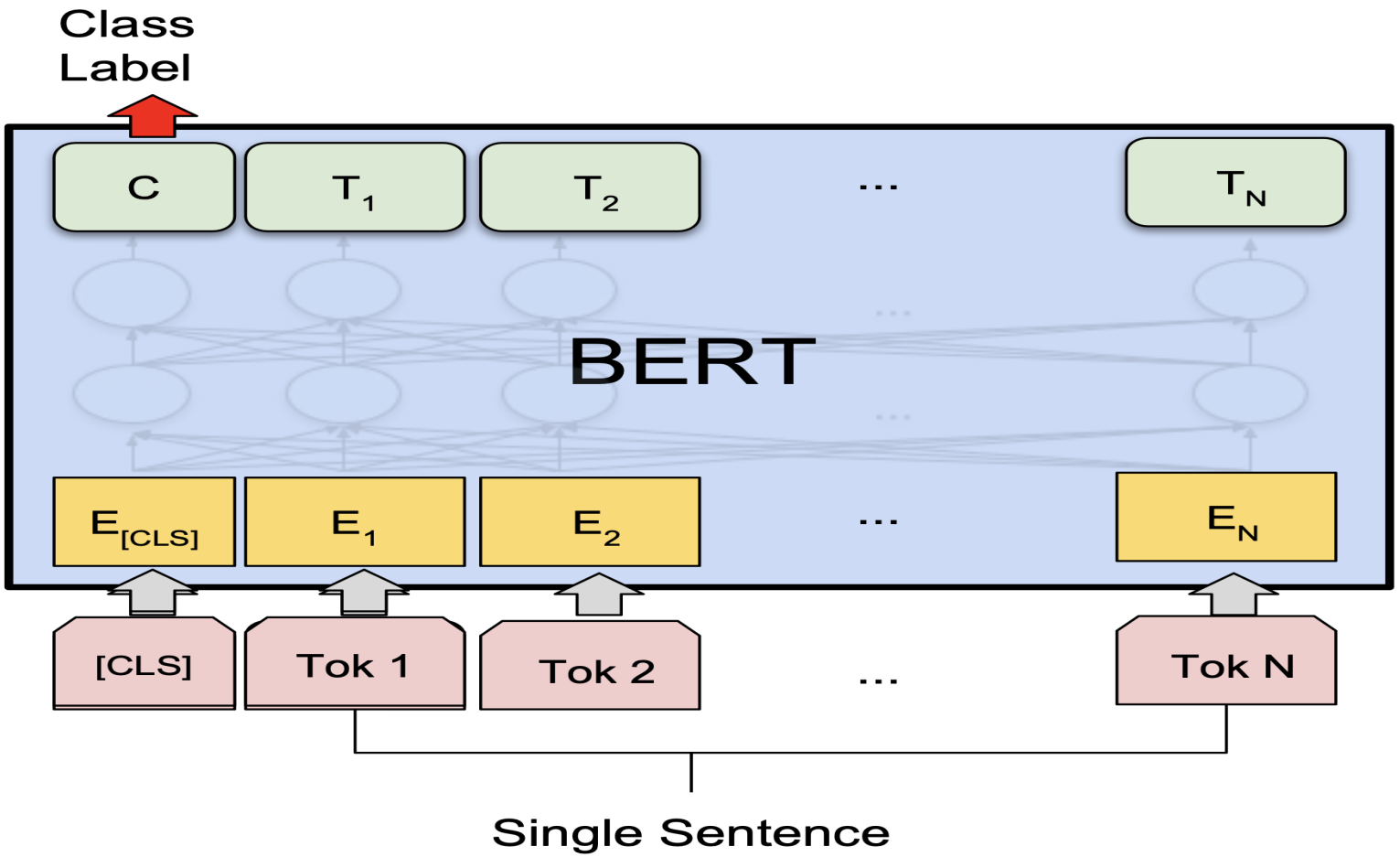
RECAP

- language modeling objective
  - masking and next token prediction
  - no explicit understanding of tasks
- this is true for encoder and decoder models
- So we need to do additional work if we want to use language models for solving tasks!

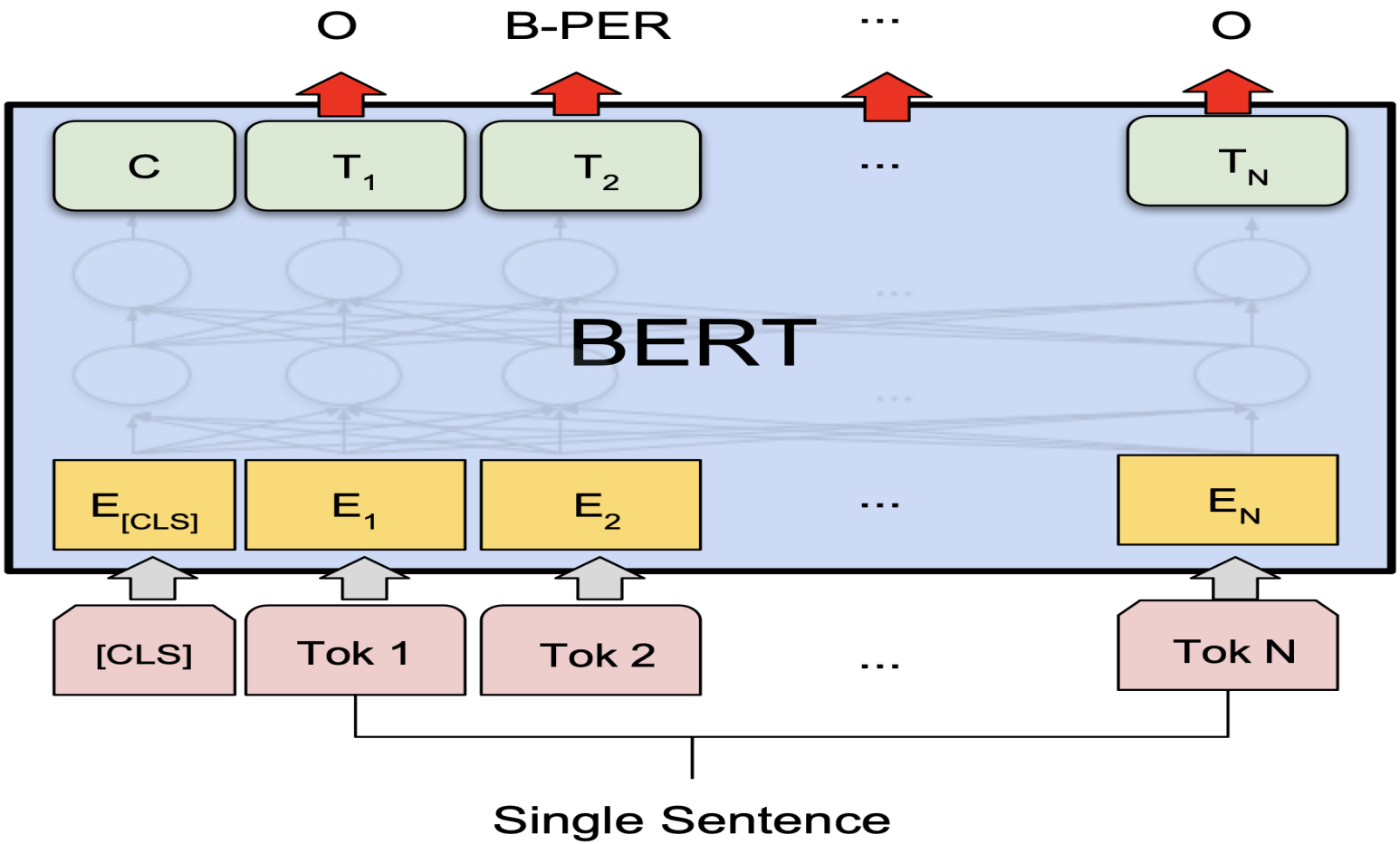
HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (1)

- "old-style" single-task fine-tuning
  - supervised training of the pretrained model on a task-specific training set of size k (where k is not small, e.g., $k = 100$)

BERT FINETUNING EXAMPLE 1



BERT FINETUNING EXAMPLE 2



HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (1)

- "old-style" single-task fine-tuning
  - supervised training of the pretrained model on a task-specific training set of size k (where k is not small, e.g., $k = 100$)
  - the output can be an arbitrary category (e.g., "0", "1" for sentiment analysis)
  - (this is the typical way encoder models like BERT are used)
  - alternatively, the output can be a meaningful "verbalizer" (e.g., "negative", "positive" for sentiment analysis) ▸ Schick et al., 2020
  - still very much relevant if you need to deploy a small efficient model for a focused task

VERBALIZER


INPUT
I really enjoyed this movie.
PATTERNS
I really enjoyed this movie. It was [MASK].
I really enjoyed this movie. Question: Is this a positive movie review? Answer: [MASK].
([MASK]) I really enjoyed this movie.
VERBALIZERS
+ = good − = bad
+ = Yes − = No
+ = positive − = negative

VERBALIZATION: SUMMARY

- Finetuning a language model on predicting meaningful labels (“good”, “bad”) requires less data than finetuning on classical NLP labels like “0” and “1”.
- Reason: The language model understands the labels already, so it doesn’t have to learn them.

HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (2)

- few-shot prompting
  - provide, in-context, k (where k is small, e.g., $k = 5$) examples of what the model is supposed to do
  - the model will then often complete the task just based on analogy to these few shots
  - this is a typical way of using current autoregressive models for tasks
  - no change to the parameters of the model, i.e., no training

FEW-SHOT PROMPTING EXAMPLE

```
1  Translate English to French:
2  sea otter => loutre de mer
3  peppermint => menthe poivrée
4  plush girafe => girafe peluche
5  cheese =>
```

(from gpt3 paper – “Translate English to French” should appear four times to make it a 3-shot example that has 3 identical shots as required by the definition of few-shot prompting)

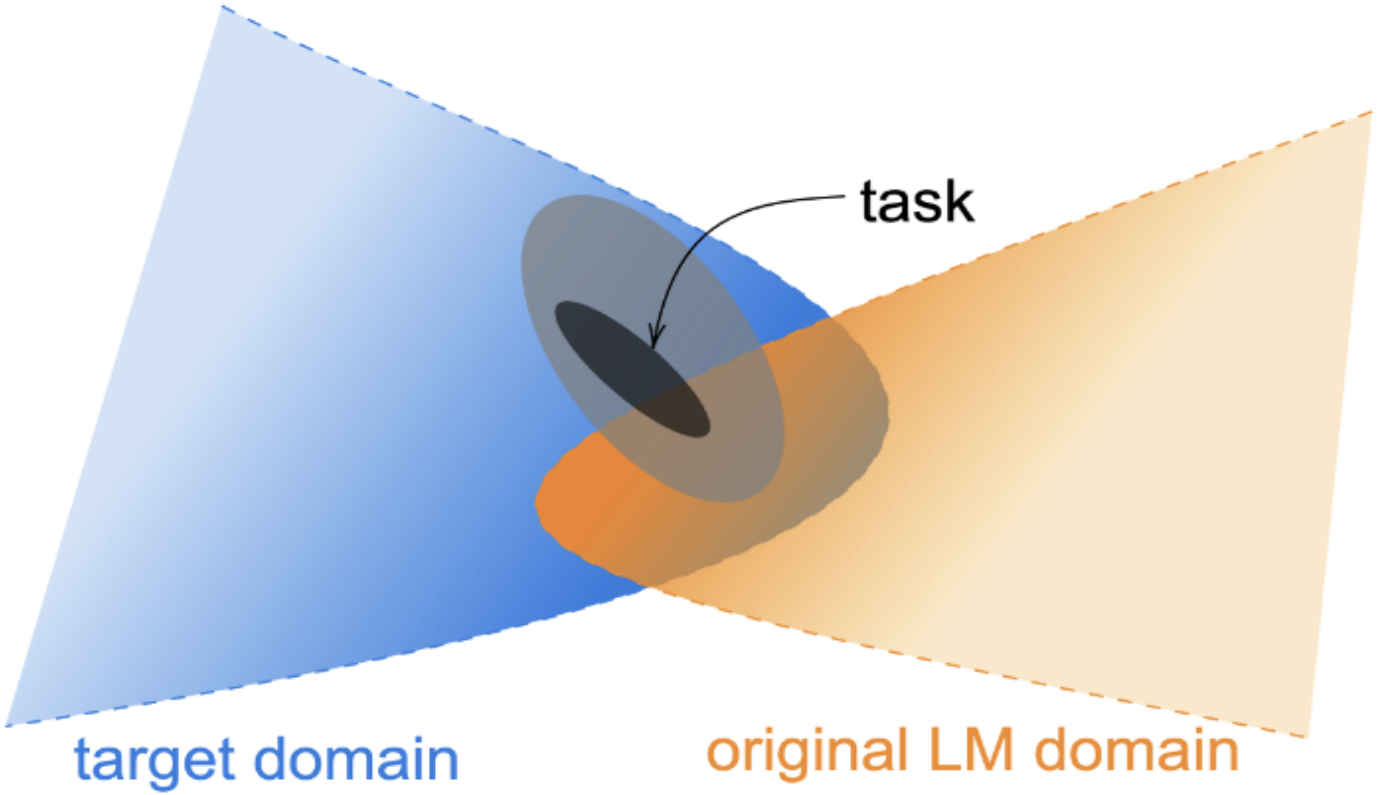
HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (3)

- multi-task finetuning
  - finetuning on a large training set of *many* tasks
  - the more tasks the better
  - method 1: consistent task format, e.g., each task is reformulated into a question-answering format
  - method 2: more open task format
  - ideally: task format includes “instructions”, similar to verbalizer
  - Examples: T0, T5 and FLAN, see below

OTHER TYPES OF FINETUNING (1)

- Finetuning has yet another meaning.
- Continued pretraining is also sometimes called finetuning.
- Continued pretraining: given a language model that has been trained on generic data (web, reddit etc), adapt it to a new domain (e.g., company-internal data) by training it on a large corpus from this new domain.
- objective: standard language modeling objective
- This results in a language model that has all the nice capabilities of a generic language model, but also understands the special domain.
- Not trivial to do well

CONTINUED PRETRAINING



TERMINOLOGY

- This lecture
  - single-task finetuning
  - multi-task finetuning
  - prompting
    (this is definitely not finetuning)
  - continued pretraining
  - **Question:** terminology clear?
- Next lecture
  - instruction tuning
    (can be seen as a form of finetuning)
  - reinforcement learning
    (can also be done through a form of finetuning)

ISSUES WITH SINGLE-TASK FINE-TUNING

- The result is a single-task model.
  - Generalization of the model is only w.r.t. to one task / data distribution
- Requires annotated data (e.g., $k = 100$)
- **Question:** Could there be other tasks that might benefit?

ISSUES WITH FEW-SHOT PROMPTING: (1) CLASSIFICATION

- Assumption: Model has learned about the task during (unsupervised) pre-training
- Write prompt so that a direct response must be given by the language model.
  - Just a label, just yes/no answer, just a name in QA
- This is not natural dialogic behavior: humans typically don't just answer with a label, yes/no, a name (although sometimes they do)
- See next lecture
- Current use of the word "prompt" is more general: basically anything you type into the LLM context window

ISSUES WITH FEW-SHOT PROMPTING: (2) TEXT GENERATION

- Again, assumption: Model has learned about the task during (unsupervised) pre-training
- Again: Write prompt so that a direct response must be given by the language model.
- Works best if the task has occurred during unsupervised training.
- **Question:** For which tasks is this expected to work well?

MULTI-TASK LEARNING: FLAN/T0/T5

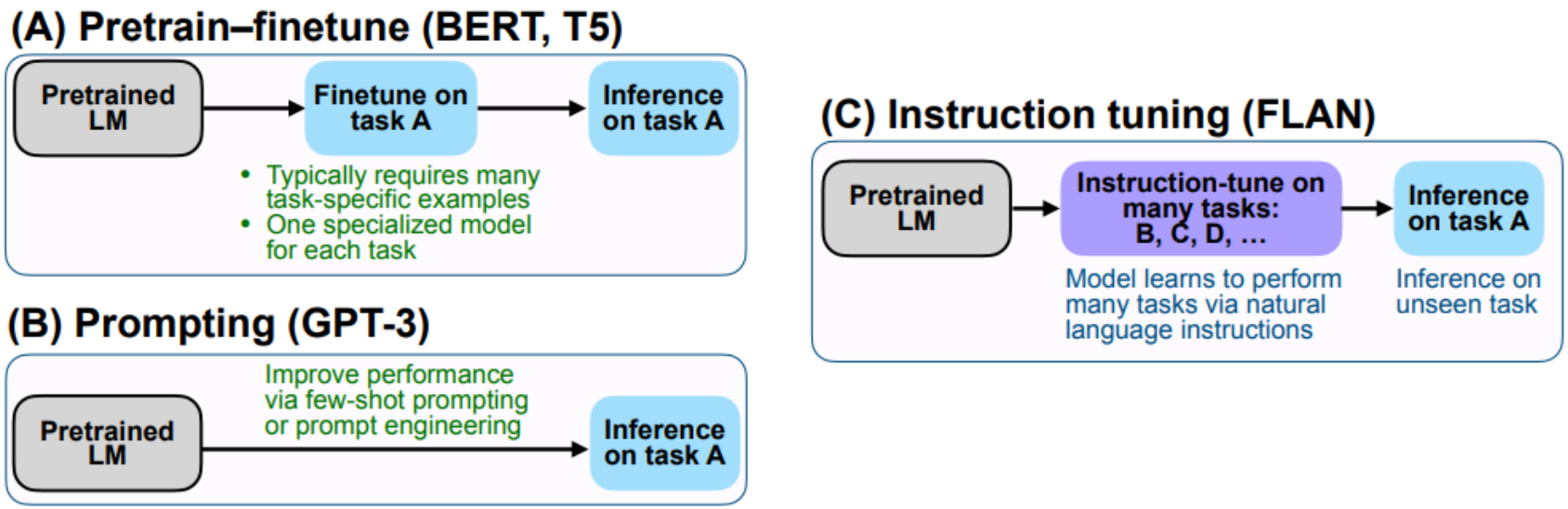
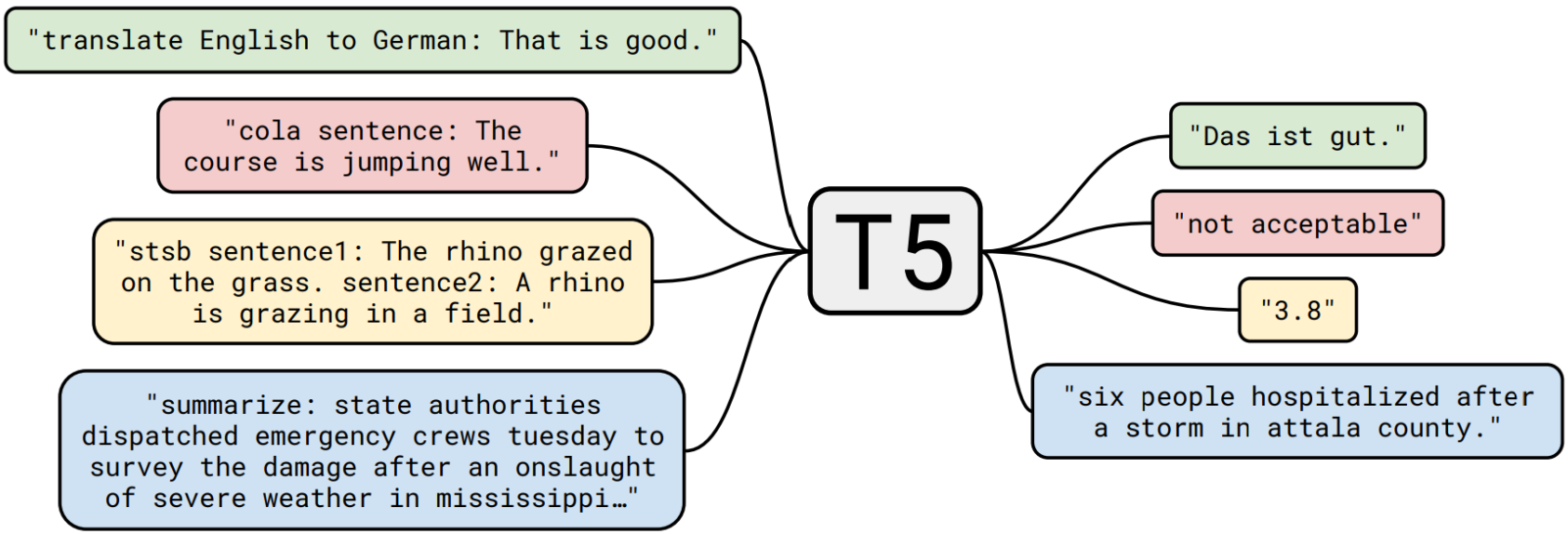


Figure 2: Comparing instruction tuning with pretrain–finetune and prompting.

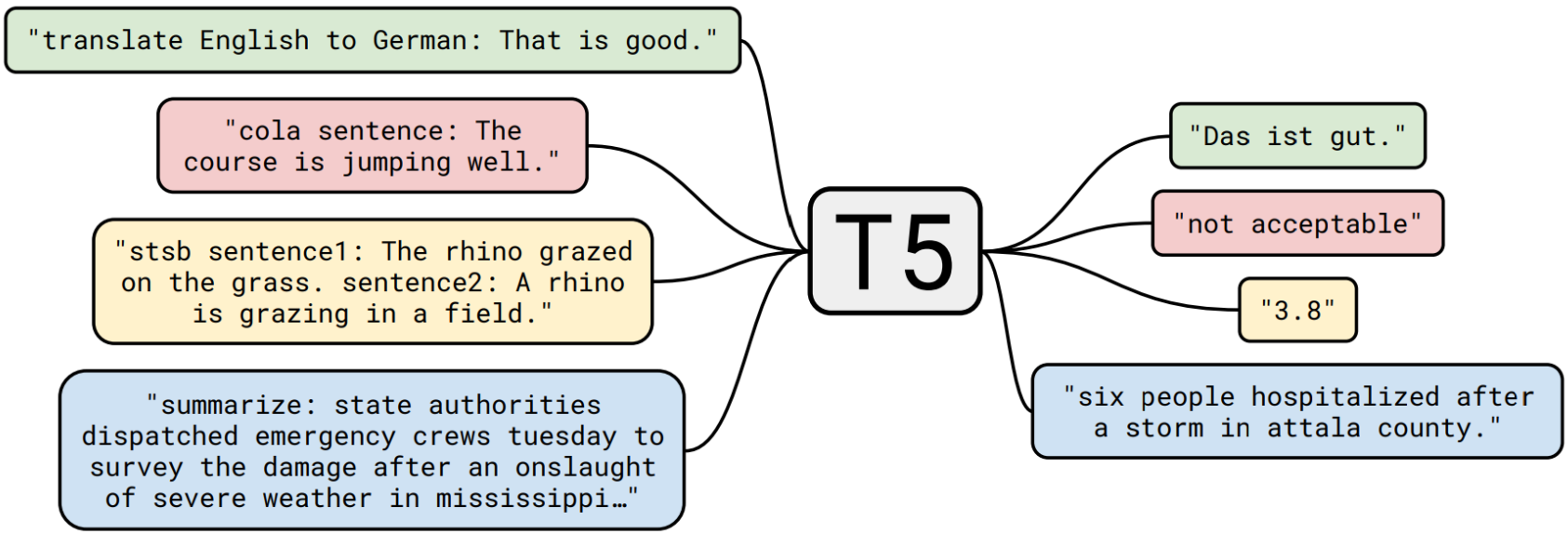
▸ Source: Wei et al., 2021

INITIAL APPROACH: MULTITASK FINETUNING: BEST OF BOTH WORLDS (FINETUNING AND PROMPTING)



▸ Source: Raffel et al., 2020

INITIAL APPROACH: MULTITASK FINETUNING: BEST OF BOTH WORLDS (FINETUNING AND PROMPTING)



▸ Source: Raffel et al., 2020

Question: What exactly does the model learn here? How well would we expect this to generalize to new tasks?

MULTITASK PROMPTED TRAINING

- Multitask Prompted Training involves learning from multiple tasks using **unified prompt formats** as a means to improve generalization to new, unseen tasks.
- (as opposed to T5: non-unified)
- This means that the model can perform well on tasks it hasn't been explicitly trained on.
- The key for this lies in the set of shared prompts it has learned from during fine-tuning.

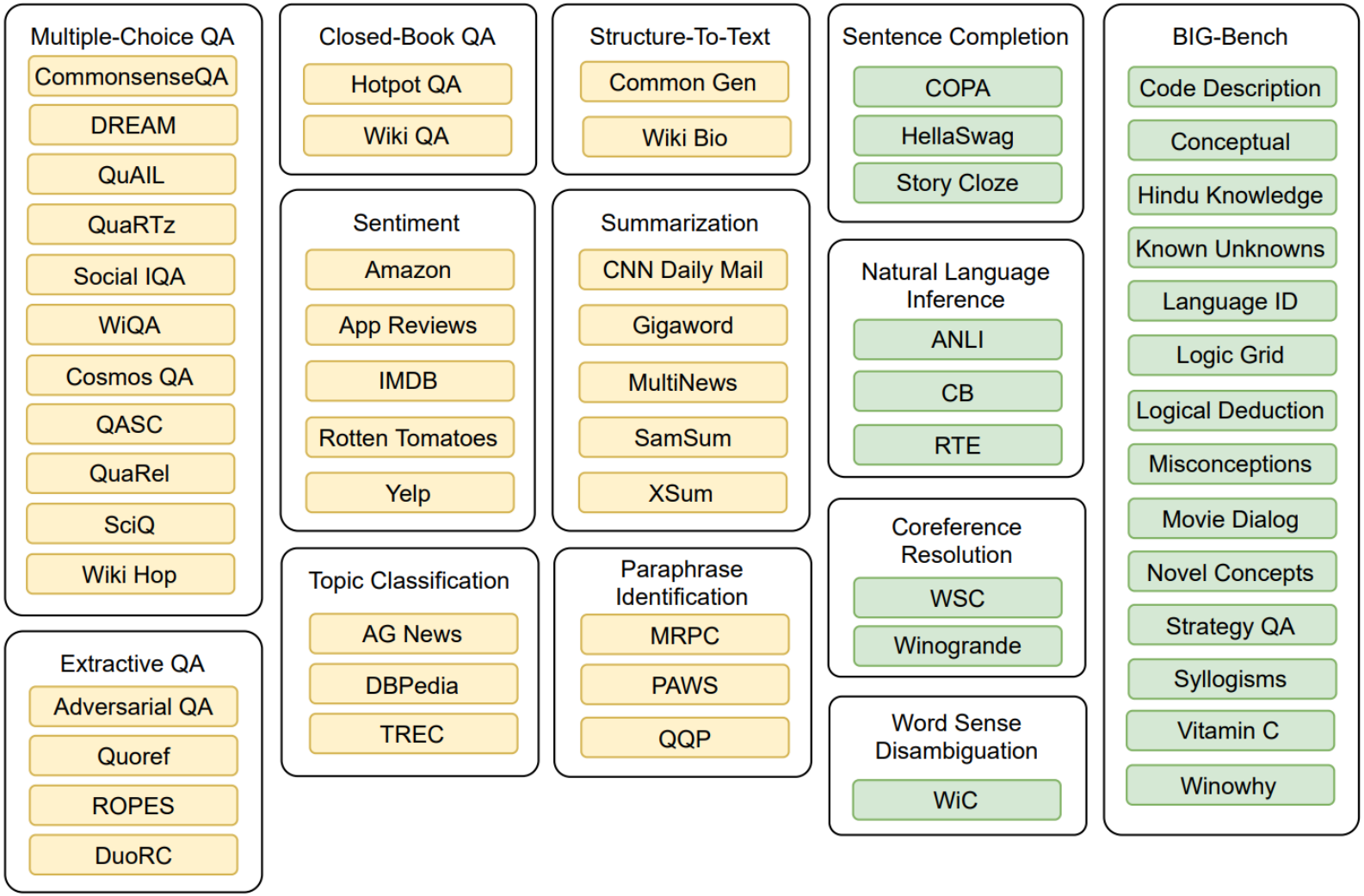
MULTITASK PROMPTED TRAINING

- Benchmark for Evaluation:
  - Instead of using held-out *samples* (as is standard in NLP)
    ...
  - ...we now use held-out *tasks*
  - All data sets belonging to a held-out task go to the test set
  - Generalization across tasks
- The “prompts” are key here to facilitate transfer to new tasks: the model can solve new tasks by generalizing from the train prompts and the ability to generate text outputs.

TYPICAL NLP TASK: COREFERENCE RESOLUTION

- One of the tasks on the next slide
- In the simplest case the task is to determine which noun phrase a referring expression like “she” or “this person” is referring to.
- Example: “Peter helped Paul because he was familiar with the problem.”
- What does “he” refer to?

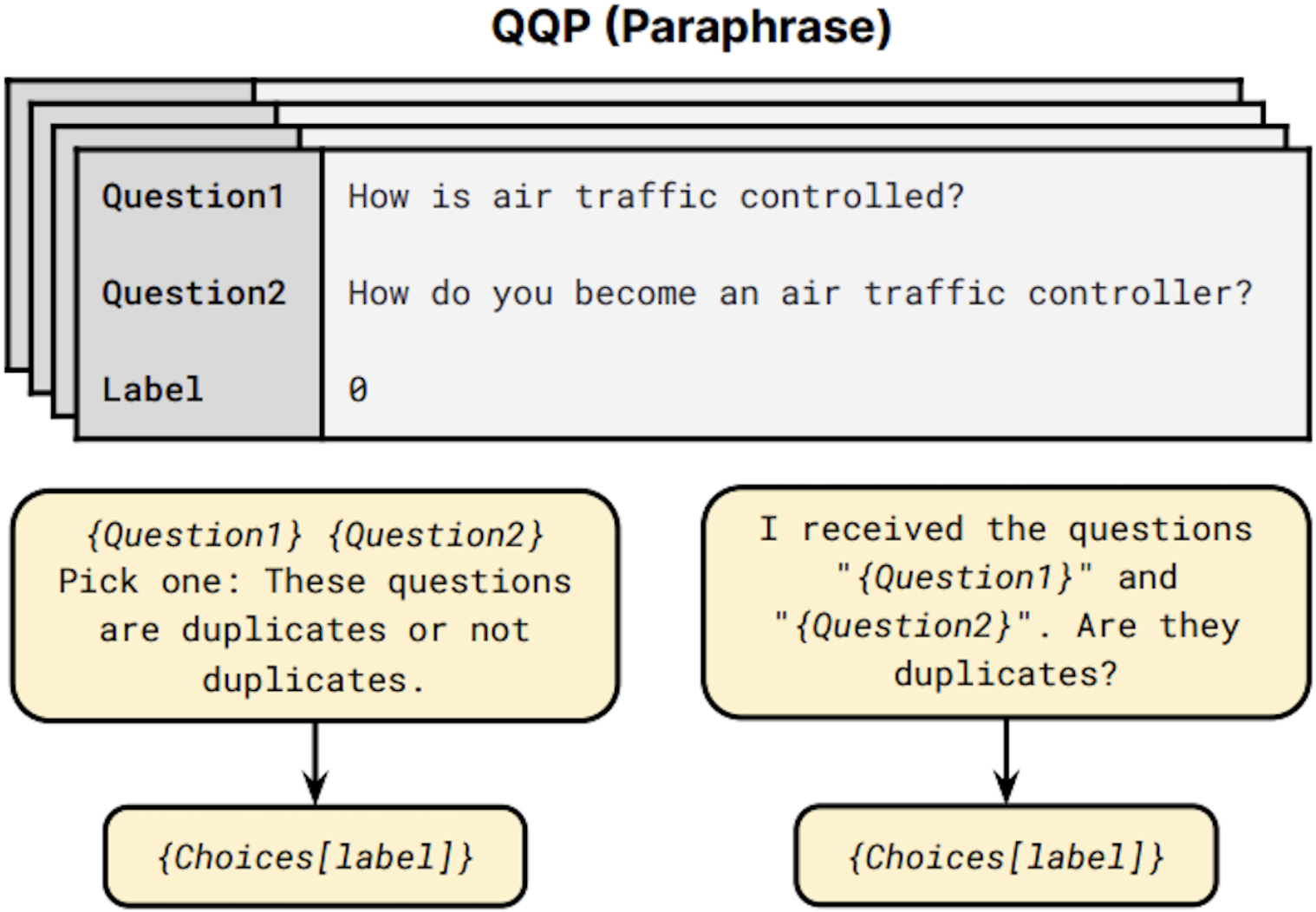
T0: TRAINING AND TEST TASKS



▸ Source: Sanh et al., 2021

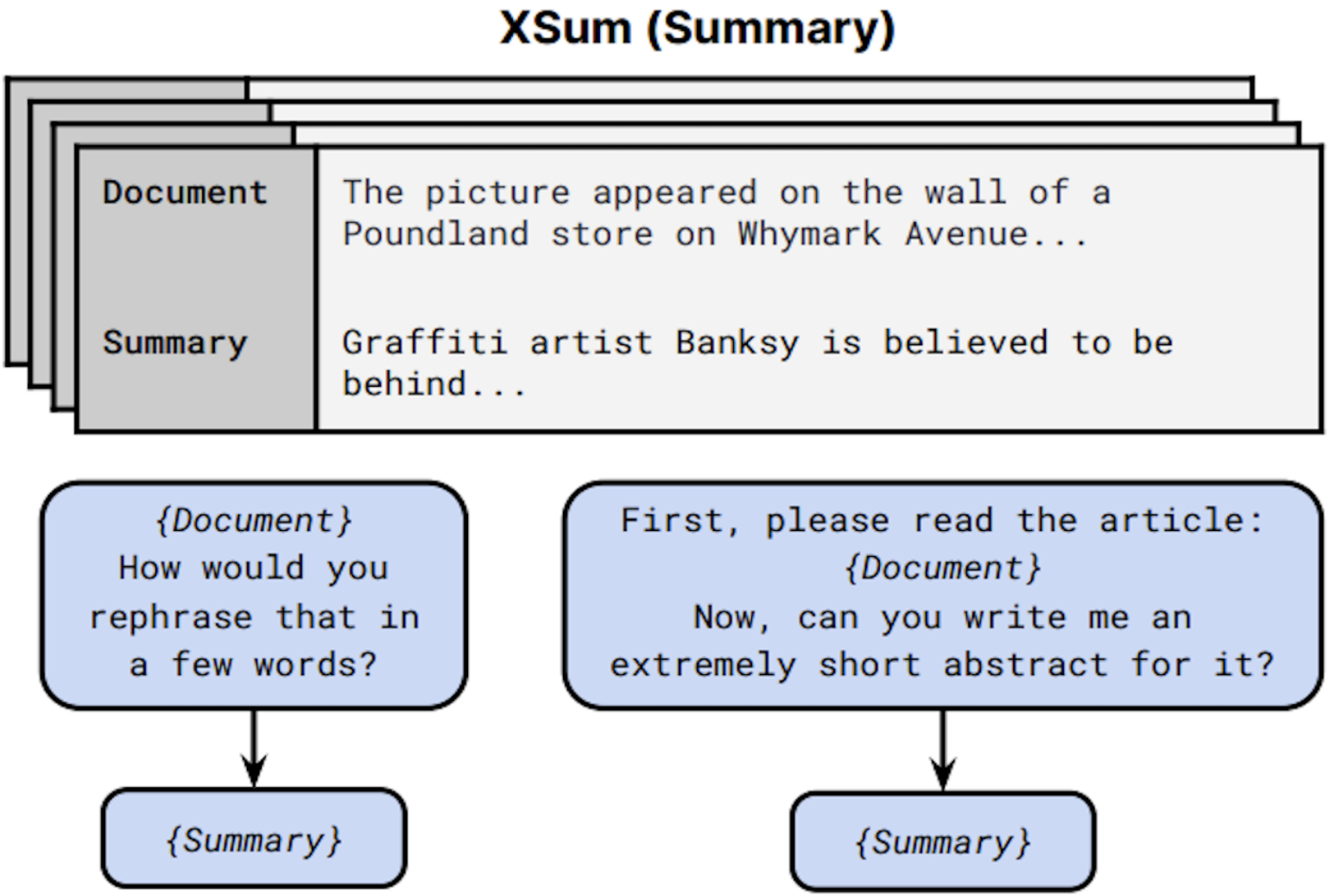
Question: What is an easy task? What is a hard task?

T0 – PROMPT TEMPLATES



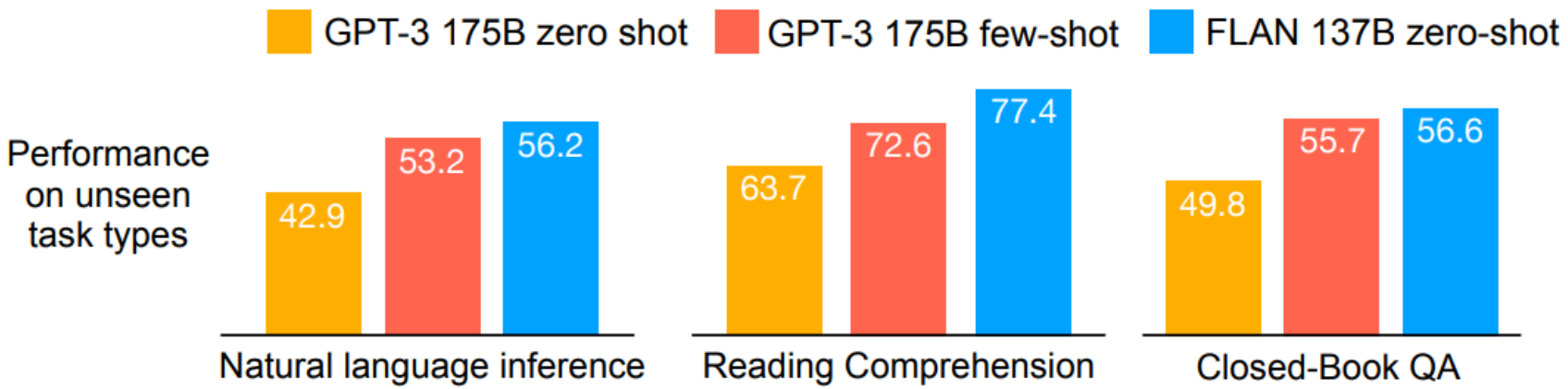
▸ Source: Sanh et al., 2021

T0 – PROMPT TEMPLATES



▸ Source: Sanh et al., 2021

EXAMPLE OF EVALUATION: FLAN



▸ Source: Wei et al., 2021

MULTI-TASK TRAINING VS INSTRUCTION TUNING

- The term “instruction tuning” was introduced by FLAN paper,
  - Source: Wei et al., 2021
- However, today “instruction tuning” refers to a wide variety of methods that train on instruction-output pairs, not just tasks
- In particular, open-ended generation and “alignment” is now included under the rubric “instruction tuning”: brainstorming, writing a joke, give me ten analogies for concept X, don’t use hate language, don’t take positions on controversial political issues etc
- These are not classical NLP tasks.
- See next lecture
- Our use of terminology in this class:
- multi-task finetuning = tasks in the classical NLP sense
- instruction-tuning: a broad approach to changing the behavior of a raw language model (i.e., trained on next-word prediction) to be “helpful, harmless and honest”. This includes training on classical NLP tasks, but it’s just one part.

FINE-TUNING CONCLUSIONS

- The more training tasks the better
- Multi-task finetuning generalizes across models
  - It works well on different architectures
- Greatly improves usability
- It is relatively compute-efficient
  - Pretraining is hugely expensive, finetuning and instruction tuning are usually much cheaper.
  - For PaLM 540 B it takes 0.2 % of pre-training compute, but improves by 9.4 %

SLIDO

- #1312409 slido

PROMPTING: DEFINITION

- Prompting always means that you do not finetune the model, i.e., you don't change the parameters of the model.
- The term prompting was initially used for when you carefully design the input to the language model, so that you will get the desired output.
- But nowadays any type of input to the model can be called a prompt.
- Few-shot or k-shot prompting means that you give a few examples or k examples of what you want the model to do as part of the input to the model.
- Zero-shot prompting means that you do not give an example. The term zero-shot prompting is often used in the context of few-shot prompting (contrasting using $k > 0$ examples vs $k = 0$ examples).